

JCMS — 2-Developer Work Split & Collaboration Roadmap

Split by **vertical ownership** (each dev owns full stack slices, not "frontend person vs backend person") — this minimizes merge conflicts, gives both devs full-stack resume credit, and lets each dev demo their own half end-to-end independently.

Dev A = "Core Ops" — case lifecycle, scheduling, courtroom/registrar/judge workflows, infra. **Dev B = "Intelligence & Access"** — auth/RBAC, documents, public portal, AI/LLM layer, real-time.

0. Shared Setup (Both Devs, Day 1 — do this together before splitting)

- Agree on stack decisions from the core blueprint (NestJS vs Spring Boot, monorepo vs polyrepo).
 - Create the **monorepo** (recommended: `apps/api`, `apps/web`, `packages/shared-types`) so both devs share TypeScript interfaces for `Case`, `Hearing`, `User`, etc. — this single decision prevents most "your API doesn't match my frontend types" pain.
 - Set up: GitHub repo, branch protection on `main`, Docker Compose (Postgres + Redis), Prisma/JPA schema skeleton, CI pipeline skeleton (lint + build on PR).
 - Agree on the DB schema from Section 2.2 of the core blueprint as the shared contract — schema changes after this point go through a PR both devs review, since both depend on it.
 - Pick one **project board** (GitHub Projects) with columns: Backlog → In Progress → Review → Done, one card per endpoint/feature.
-

1. Ownership Split

Area	Dev A — Core Ops	Dev B — Intelligence & Access
Auth & RBAC	—	✓ JWT auth, role guards, Keycloak/Auth0 setup
Judge Dashboard	✓ Docket view, disposition, courtroom mgmt	—
Advocate Portal	✓ e-filing, tracking, calendar	(consumes Dev B's document upload API)
Registrar Portal	✓ Verification, queue mgmt, emergency override	—
Public Portal	—	✓ Cause list, case-status lookup
Priority Scheduling Engine	✓ Weighted algorithm, Redis sorted-set queue	(Dev B's AI urgency estimator plugs into this as an input)
Document Upload/Storage	—	✓ S3/MinIO, versioning, checksum, virus scan job
Audit Log (hash-chained)	✓ Core implementation	—
WebSocket Real-Time Layer	(emits events from own domain)	✓ Socket.IO infra, room management, client hooks
AI: Petition Extraction + OCR	—	✓
AI: Priority Urgency Classifier	(defines the scoring interface it must feed)	✓ Implements against Dev A's interface
AI: RAG Summarization + PGVector	—	✓
AI: Transcript → Draft Order	—	✓
DevOps/Infra (CI/CD, Docker, IaC)	✓ Owns overall, both contribute	—
Frontend design system (shared components)	✓ Sets up shadcn/ui base + shared layout	Both extend for their own portals

Why this split works: Dev A's surface area (case/hearing/scheduling/audit) is entirely deterministic backend logic — great for someone who wants to lead with "systems design +

algorithms" on their resume. Dev B's surface area (auth, documents, AI, real-time) is where all the flashy differentiators live — great for someone who wants to lead with "applied AI + real-time systems." Both end up with a full-stack, independently-demoable slice.

2. Integration Contracts (define these together, before building in parallel)

The two verticals only touch at a few seams — nail these down explicitly in Week 1 so both devs can build independently afterward without blocking each other:

1. **Shared TypeScript types** (`packages/shared-types`) for `Case`, `Hearing`, `User`, `Document`, `PriorityFactor` — whoever needs a field first proposes it, both approve via PR.
2. **Priority score interface** — Dev A owns the deterministic scoring engine; Dev B's AI urgency classifier must output `{ urgency_class, rationale, suggested_weight_delta }` matching an interface Dev A defines. This is the single most important seam — agree on it explicitly before either side starts building.
3. **Auth middleware** — Dev B builds it once; Dev A's routes just apply the `@Roles()` guard/decorator Dev B exposes.
4. **WebSocket event contract** — Dev B owns the Socket.IO server; Dev A's domain (queue reorder, hearing scheduled) just calls `emitEvent('queue:updated', payload)` against an interface Dev B defines.
5. **Document metadata contract** — Dev B owns upload/storage; Dev A's case views just consume a `DocumentSummary` type to link documents to cases.

Put all 5 of these in a `CONTRACTS.md` file in the repo root, versioned like an API spec.

3. Git Workflow for 2 Devices

- **Branching:** `main` (protected, deploy-ready) → `dev-a/*` and `dev-b/*` feature branches → PR into `main`. No direct pushes to `main`.
- **PR review:** the *other* dev reviews every PR that touches a shared contract (schema, types, auth, WebSocket events, priority interface). Purely internal changes within your own vertical can self-merge after CI passes, to keep velocity up.
- **Daily sync (15 min async or live):** what merged, what's blocking, any contract changes needed.
- **Schema migrations:** whoever needs a schema change opens a PR with the migration file; the other dev must approve before merge, since both depend on the same DB.
- **Conflict avoidance:** because ownership is split by vertical (not by file type), the two devs rarely touch the same files — this is the main reason to prefer this split over "frontend dev / backend dev."
- **Environment parity:** both devices run the same Docker Compose stack (Postgres, Redis, MinIO) so local dev matches prod-like behavior — avoids "works on my machine" drift between two devices.

4. Phase-by-Phase Split (maps onto the Phase 1-3 roadmap from the core blueprint)

Phase 1 — MVP (Weeks 1-6)

Week	Dev A	Dev B
1	Shared setup (together) + DB schema + Case/Hearing CRUD	Auth (JWT + roles) + User model
2	Advocate e-filing flow	Document upload (local storage first, S3 later)
3	Registrar verification + manual scheduling	Public case-status lookup + static cause list
4	Judge docket view + disposition recording	Notification system (email)
5	Basic audit log	Frontend shared component library + auth-gated routing
6	Integration week (together): wire both halves, fix contract mismatches, deploy Phase 1 demo	

Phase 2 — Core Differentiators (Weeks 7-12)

Week	Dev A	Dev B
7-8	Weighted priority engine + Redis sorted-set queue	WebSocket infra + live docket updates
9	Emergency override workflow (state machine)	PGVector setup + document chunking/embedding pipeline
10	Hash-chained audit log upgrade	RAG summarization ("Summarize File" feature)
11	PDF cause-list generation job	AI priority urgency classifier (feeds Dev A's engine via the agreed interface)
12	Integration + testing week (together)	

Phase 3 — Advanced/Cloud-Ready (Weeks 13-18)

Week	Dev A	Dev B
13	Kubernetes/ECS manifests, Terraform	AI petition extraction (OCR + structured output)
14	CI/CD pipeline to real cloud env	AI transcript → draft order (Whisper + diarization)
15	Observability (Prometheus/Grafana, Sentry)	Elasticsearch/OpenSearch migration for public search
16	Load testing (k6) on scheduling/queue endpoints	Eval harness for AI extraction accuracy (labeled test set)
17	Security hardening pass (rate limiting, dependency scan)	Prompt versioning + cost/latency tiering cleanup
18	Together: demo video, architecture diagram, README, final polish	

5. Individual Resume Framing (once complete)

- **Dev A:** "Designed and implemented a weighted priority-scheduling algorithm backed by a Redis sorted-set queue, a hash-chained tamper-evident audit log, and a full case-lifecycle backend, deployed via Kubernetes/Terraform with CI/CD."
- **Dev B:** "Built a full RBAC/auth layer, real-time WebSocket architecture, and a RAG-based legal document summarization pipeline (PGVector + LLM), including AI-assisted petition extraction and speech-to-text draft order generation."

Both narratives are independently strong and independently verifiable in an interview — which is the actual goal of this split, beyond just parallelizing the work.